

Mapping Dynamic, Hierarchical Quantum Circuits

Anonymous Author(s)

Abstract

Qubit mapping is a critical pass in quantum compilation. Despite various advances, dynamic circuits, those exhibiting data dependent control-flow, often resulting from qubit measurements, are not yet supported by the vast majority of available qubit mappers. The crucial limitation to overcome is the dependence on flat, one-dimensional representations of circuits. Further, qubit mappers currently lack compiler abstractions that capture the hierarchical nature of circuits, hindering the qubit mapping process.

In this paper, we introduce a new qubit mapping method and analyses to tackle hierarchical dynamic circuits. Our novelty resides in four key aspects: modeling (statically) sub-circuits in disjoint control-flow paths, introducing a novel Qubit Reconciliation pass to maintain consistency between sub-circuit and control-flow boundaries, a loop-entry remapping pass, and a refined cost function enhanced for SWAP count, circuit depth, circuit latency and error. We demonstrate the efficiency of our approach on a wide range of dynamic circuits on two monolithic Quantum Processing Units of 127 and 156 qubits, and on chiplet hexagon-based QPUs. On monolithic QPUs, our qubit mapper improves the SWAP count by up to 52%, depth by up to 18%, latency by up to 18.6%, and error by up to 40%. On chiplet architectures, we achieve improvements of up to 36% on SWAP count, 8.7% on depth, 15% on latency, and 15% of error.

Keywords: Hierarchical Circuits, Qubit Mapping, Dynamic Circuits, Reconciliation.

ACM Reference Format:

Anonymous Author(s). . Mapping Dynamic, Hierarchical Quantum Circuits. In *Proceedings of* . ACM, New York, NY, USA, 15 pages.

1 Introduction

Recent advances in technology have enabled mid-circuit measurements and qubit reset in quantum systems [12, 43]. Such features are fundamental in enabling new classes of experimental methods such as Repeat-Until-Success, classical feed-forward control, Quantum Error Correction, and Fault-Tolerant Quantum Computing. This paper tackles the problem of qubit mapping and routing ~~for dynamic circuits (circuits with data-dependent control-flow), where mid-circuit measurements are leveraged to condition the execution of gates forming sub-circuits.~~ ~~for circuits with nested, data-dependent control-flow, where the result of mid-circuit measurements are used to select the execution of gates forming sub-circuits.~~ We target classes of circuits that exhibit selective and iterative (loop-based) control-flow constructs

with fixed-count, symbolic-count and nested control-flow structures.

Dynamic control-flow is the principal inhibitor of current qubit mapping methods, as it induces execution uncertainty and a dynamic dependence structure in quantum circuits. Multiple (new) classes of applications use mid-circuit measurements to condition the execution. Examples include teleportation and fault-tolerant protocols, *magic state distillation*, and phase correction. Teleportation protocols leverage classical feedback using circuits of constant depth that generate multi-partite entanglement [23]. Additionally, dynamic control-flow also enables fault-tolerant protocols, which often rely on feed-forward mechanisms.

There is a critical demand for effective qubit mapping methods that support *quantum circuits with data-dependent control-flow* (dynamic circuits, for short), as evidenced by recent work [3, 7, 9, 12, 15, 17, 21–23, 30, 31, 44–46, 50, 54, 57, 66]. While there has been a significant body of work that maps and routes qubits to address nearest-neighbor topological constraints [1, 28, 33, 52, 55, 60, 61, 63, 64], these have yet to tackle the problem in the context of dynamic control-flow. The primary underlying challenge is that the qubit mapping process is an inherently straight-line code compilation pass. To repair the connectivity between qubit arguments of any two-qubit gate, the current logical-to-physical mapping must be known. In light of this, naive mapping strategies for loops with fixed – known *a priori* – iteration counts resort to fully unrolling the loop body, effectively producing a static circuit. However, for-loops with parametric (symbolic) counts, while-loops and branch conditionals with data-dependent control-flow (using classical bits obtained from mid-circuit measurements) pose significant challenges to qubit mappers.

In this paper, we introduce DynamiQ, a novel multi-objective qubit mapping and routing framework designed and implemented to handle dynamic quantum circuits featuring data-dependent control-flow. Unlike prior methods designed solely for static, acyclic circuits, DynamiQ explicitly models control-flow structure, ensuring consistent mapping states across all possible execution paths. It also introduces a fundamentally improved hardware-, error-, and latency-aware cost function. DynamiQ’s cost model accounts for the current circuit depth and integrates QPU calibration data (two-qubit error rates). These additions make the mapping method both error-resilient and depth-efficient, improving mapping quality for dynamic circuits.

The contributions of our work are: i) A new hierarchical qubit mapping method ~~that unifies the qubit mapping state across disjoint execution paths of the same nesting level~~

by applying the token-swapping approximation of [32] at control-flow boundaries. The new method supports fixed and symbolic iterative constructs and multi-branch conditionals that exhibit data-dependent control-flow; ii) An enhanced multi-objective mapping cost function for straight-line circuits that builds on the Qlosure method [4]. The new cost function unifies four objectives, circuit depth, SWAP count, circuit error (i.e. aggregated error) and circuit latency; iii) Due to the lack of quantum benchmark suites with dynamic control-flow, we develop a new generator for hierarchical circuits, d-QUEKO (Dynamic-QUEKO) based on the QUEKO algorithm [56]; iv) Finally, an evaluation showing the effectiveness of DynamiQ over the LightSABRE mapper [68], on two monolithic QPUs (127/156-qubit) and two chiplet architectures, on dynamic and rotated surface-code circuits.

2 Motivation

Numerous mapping methods have been proposed to tackle the qubit mapping and routing problem for *flat static circuits* [1, 5, 8, 28, 34, 35, 38, 40, 42, 52, 53, 55, 61, 62, 69]. However, qubit mapping and routing for dynamic circuits (i.e. exhibiting data-dependent control-flow conditions such as $b[0] == 0$) remains an open problem. Work has been limited to producing a flattened program representation via loop unrolling [17] and analyzing the Directed Acyclic Graph (DAG) representation to achieve SWAP reduction by qubit reuse [24]. Despite such advancements, mapping methods based solely on DAG analysis lack the ability to handle hierarchical circuits. This is because they lack composable compiler abstractions that allow us to overcome the limitations inherent in mapping straight-line circuits (those expressible as DAGs). Compositional abstractions, such as those found in affine frameworks [4, 18] are key to model, extract, and leverage circuit information outside of a single circuit level.

Data Dependent Control Conditions. A common assumption throughout qubit mapping methods is that all gates always execute. The presence of data-dependent conditions that control subsets of gates within iterative and selection constructs poses crucial challenges to mapping methods; namely, it introduces qubit mapping state gaps. Not knowing the exact logical-to-physical qubit mapping at any given point brings the mapping algorithm to a halt. Such qubit mapping state gaps originate from disjoint control-flow paths, or different sequences of gates being executed in each sub-circuit, leading to multiple qubit mapping states.

Sub-Optimal Mappings Even without Multiple Control-Flow Paths. Qubit mapping methods, even those that target circuits represented as DAGs, are currently limited to heuristics based on graph traversals, which result in mapping decisions made with local information. As a result, today’s heuristics are still far from the optimal solution [55]. Having multiple control-flow paths resulting from data-dependent conditions further reduces the effectiveness of all qubit mapping methods, which assume that all gates execute.

The Key Contribution. Our key scientific contribution is a mapping algorithm designed for hierarchical circuits that exploits the transitive closure of dependence relations at each level of the hierarchical circuit. This information is used to prioritize mapping decisions. In addition, to support nested and disjoint/distinct control-flow paths taken by dynamic conditions we leverage a novel *Qubit Reconciliation Pass* whose task is to convert the qubit mapping state between entry and exit points of hierarchical circuits into a *known qubit steady state* at minimal cost.

Table 1. Support for mapping circuits with dynamic control-flow and multiple control-flow paths. Objective types: A*-Search (A*S), Bounded Distance (BD), Critical-Path Aware (CPA), Depth (D), Depth-Rate (DR), Error (E), Qubit Fidelity (QF), Swap Minimization (SM), Transitive Dependence Volume (TDV). Dyn. = Dynamic, Hier. = Hierarchical, Par. = Parametric, Ent. = Entry Remapping, Crit. = Critical Path.

Method	Dyn.	Hier.	Par.	Ent.	Crit.	Objectives
QISKIT Sabre [28]	✓	✓	✗	✗	✗	E, D
TKET [53]	✗	✗	✗	✗	✗	E, BD
QMAP [60]	✗	✗	✗	✗	✗	BD, A*S, SM
Cirq [1]	✗	✗	✗	✗	✗	QF
Qlosure [4]	✗	✗	✗	✗	✓	TDV
This work	✓	✓	✓	✓	✓	TDV, E, CPA, DR

We summarize the support of mapping methods in Tab. 1. Only QISKIT LightSABRE [68], the enhanced SABRE method [28] used by QISKIT, supports circuits that are both dynamic and hierarchical (with nested control structures). Mapping is performed recursively, but optimizations cannot cross control-flow boundaries. DynamiQ is different from QISKIT’s LightSABRE in four ways. First, LightSABRE simply pads additional branches with SWAPs to maintain the inherited logical-to-physical qubit mapping. In contrast, DynamiQ maps each sub-circuit and selects the one with the lowest mapping cost as the steady-state, padding all other branches with the needed SWAP operations. Second, due to the dynamic nature, LightSABRE adopts a *look through* look-ahead policy that considers control-flow operations as 1-qubit gates; In this regard, DynamiQ views the remaining circuit as a static multi-qubit gate. Third, our method considers the refinement of the inherited mapping, allowing to further improve the overall cost. Finally, whereas LightSABRE prioritizes gates by how close their operands currently are on the device, DynamiQ weighs gates by how many two-qubit operations depend on it, directly or indirectly, computed as a transitive closure over the dependence relations at each level of the hierarchy. Other mappers do not support these features despite relying on several types of objectives. None of the mappers support qubit state remapping at juncture points to better optimize across hierarchical boundaries. As we will see in the main evaluation, our updated cost function (Sec. 5.4) is quite effective in monolithic and modular QPUs, and is capable of trading SWAP counts for error in the presence of expensive

QPU links. We also note that we enhance the Qlosure mapping method [4] and use it to map static sub-circuits. The key differences w.r.t [4] are the *qubit fidelity* and *critical path awareness* extensions described in Sec. 5.4

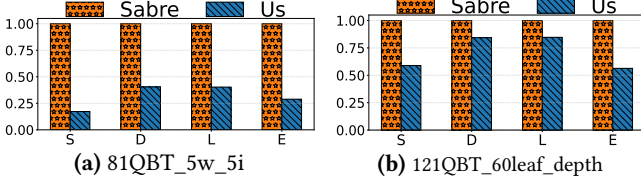


Figure 1. QISKIT Sabre vs. DynamIQ on d-QUEKO circuits (Sec. 6) on the IBM Brisbane QPU. Y-axis shows *normalized values* (lower is better) of four metrics: SWAPs (S), Depth (D), Latency (L), and Error (E). Fig. 1a presents 5-nested loops, while the one on the right Fig. 1b presents five back-to-back loops interleaved with regular gates.

To conclude our motivation, Fig. 1 shows excerpts of our results. Due to the lack of mappers capable of supporting dynamic circuits, we compare our mapping method with the SABRE implementation in QISKIT. The effectiveness of our mapper is established across four metrics: SWAP count (S), circuit Depth (D), circuit Latency (L), and circuit Error (E). As we can see, our proposed method achieves notable improvements on dynamic circuits, where QISKIT Sabre is the only mapper capable of handling such circuits.

3 Background

Quantum Computing Fundamentals. Quantum Processing Units (QPUs) compute using *qubits*, which exist in a linear superposition of the computational basis states $|0\rangle$ and $|1\rangle$. Qubit states are transformed via unitary operations known as *quantum gates*. Single-qubit gates (e.g., H) act on individual qubits, while two-qubit gates (e.g., CNOT, SWAP) act on a qubit pair. QPUs are composed of physical qubits connected through a *coupling graph* $G_{hw} = (Q_{phys}, R_{hw})$, where each vertex $q_i \in Q_{phys}$ is a qubit and each edge $(q_i, q_j) \in R_{hw}$ denotes a physical link. Two-qubit gates can be executed directly only if their operands share an edge in G_{hw} . Besides unitary gates, a *measurement operation* $m(q, c)$ collapses the state of a qubit q onto the computational basis and stores the outcome into a classical register c , where it stays readable by the control electronics for the rest of the execution [13].

Gate Errors and Fidelity. In current QPUs, each gate has an *error rate* ϵ , often expressed through its *fidelity* $F = 1 - \epsilon$, where ϵ denotes the probability of failure. As each link (q_i, q_j) in the graph G_{hw} has its own fidelity, differentiating among them leads to significant qualitative improvements. Moreover, two-qubit gates are an order of magnitude noisier than single-qubit operations, and dominate the overall circuit fidelity, making topology- and fidelity-aware compilation crucial for reliable execution.

Dynamic Circuits and Classical Control. Recent QPUs support *dynamic circuits*, which allow to condition gates to measurement results. This enables data-dependent control through constructs such as if-else branches, while loops, and conditional gate execution. For instance, a qubit measurement can control the execution of a later gate or the execution of a repeat-until loop once a measurement outcome is observed. Such feedback mechanisms make permit to implement adaptive algorithms, error mitigation routines, and mid-circuit corrections directly on hardware. Fig. 2a illustrates the latter case: qubit q_3 is measured into the classical bit c_0 , and the loop body repeats while $c_0 = 0$. Since the body re-measures q_3 , the number of iterations is decided at run-time and is unavailable to the compiler.

Quantum Routing Problem. Quantum circuits are defined over *logical qubits*, while hardware executes operations on *physical qubits*. The compiler maintains a mapping $\varphi : Q_{logical} \rightarrow Q_{phys}$ that assigns each logical qubit to a physical one on the device. A two-qubit gate acting on logical qubits (q_i, q_j) is *feasible* only if the corresponding physical qubits $(\varphi(q_i), \varphi(q_j))$ share an edge in G_{hw} . If not, the compiler inserts *SWAP gates* along a path to move qubit states until the pair becomes adjacent.

Example: Consider a linear hardware topology with physical qubits p_0, p_1, p_2, p_3 connected as $p_0-p_1-p_2-p_3$. Suppose a CNOT acting on qubits q_A and q_D , initially mapped to p_0 and p_3 , which are not directly connected. To make them adjacent, the compiler applies a sequence of SWAPs: $SWAP(p_0, p_1)$, and $SWAP(p_1, p_2)$. This makes the logical qubits q_A (in p_2) and q_D adjacent, allowing the CNOT to be executed.

The *quantum routing problem* consists of determining where and when to insert SWAP gates that make operands of two-qubit gates physically adjacent, while minimizing the circuit depth and noise, avoiding swaps on low-fidelity edges or moves that place qubits on noisy links, as such choices increase circuit errors.

4 Overview

We now describe how DynamIQ handles dynamic circuits. We lift every quantum program into an *affine-a hierarchical* dependence graph, where each gate, measurement, or control-flow construct is modeled as a node. Edges capture dependencies between gates sharing at least one logical qubit. Fig. 2 shows this process in a circuit with a while-loop; the loop body is abstracted as a high-level node in the outer DAG, but internally it represents a subgraph corresponding to one iteration of the loop. In this hierarchical view, a dynamic block behaves as a *multi-qubit gate* (of all accessed qubits inside the block). For example, a loop body containing gates $CNOT(q_1, q_2)$ and $CNOT(q_5, q_3)$ is represented as a single node that uses qubits $\{q_1, q_2, q_3, q_5\}$, thereby preserving dependence edges between the block and other operations.

Mapping Strategy for Dynamic Blocks. Because execution paths in dynamic circuits cannot be determined at

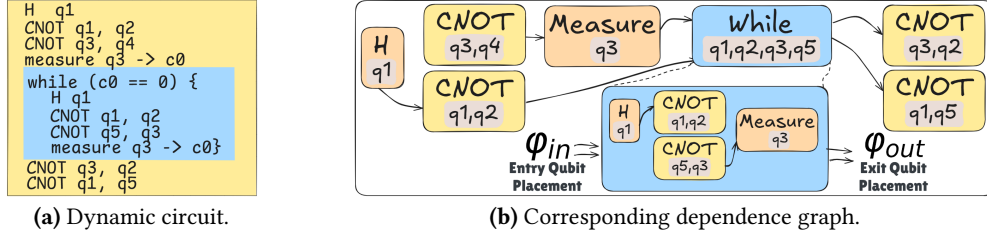


Figure 2. Dynamic circuits as hierarchical dependence graphs: dynamic blocks (loops, conditionals) form high-level nodes with subgraphs, compiled independently by DynamiQ with fixed entry/exit mappings.

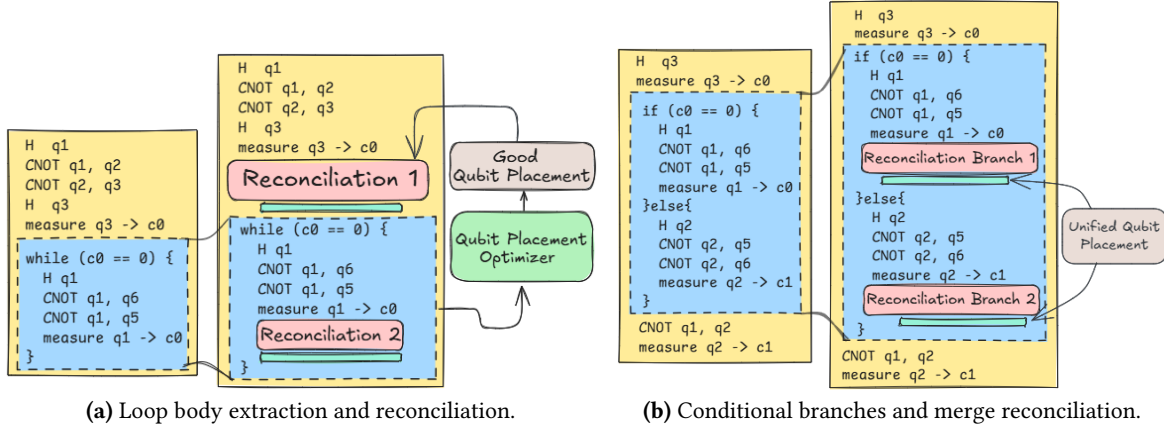


Figure 3. Reconciliation strategies in DynamiQ: (a) loops learn an entry mapping φ_{in}^* for consistent placement; (b) conditionals reconcile branch mappings at merge points for deterministic execution.

compile-time, DynamiQ imposes a key constraint: **each dynamic block is compiled with a fixed entry and exit qubit placement**. Every block B is associated with a deterministic pair $(\varphi_{in}^B, \varphi_{out}^B)$, where φ_{in}^B denotes the *logical-to-physical qubit mapping* at the point of entry into the block (i.e., how logical qubits are placed on hardware before the block executes), and φ_{out}^B denotes the corresponding mapping upon exiting the block after compilation. This invariant ensures that, regardless of how many loop iterations execute or which branch is taken, the compiler always returns to a predictable placement state, enabling deterministic continuation of the mapping process.

To enforce these invariants, DynamiQ inserts *reconciliation passes* that restore a target mapping between blocks: for a loop, a reconciliation pass is invoked for the sub-circuit associated to each distinct control-flow path at the end of each iteration so that execution always returns to φ_{in} ; for a conditional block, reconciliation gates are appended to each branch to merge all exit mappings into a common φ_{out} . More details are given in Sec. 5.5.

The DynamiQ mapping algorithm operates recursively: whenever it encounters a dynamic node in the DAG, DynamiQ recurses into the node’s sub-graph (e.g., the while-loop in Fig. 2), applies its mapping and reconciliation logic, and then resumes global routing with a known consistent placement. At runtime, whichever branch or iteration path the quantum circuit follows, the precomputed SWAPs ensure that qubit placements remain valid.

5 Mapping Dynamic Circuits

5.1 Considerations of Mapping Control-Flow Blocks

DynamiQ supports structured quantum programs using loops and conditional statements. Our approach works entirely at compile-time, and supports nested control-flow structures, including fixed-count and parametric loops (for-loops and while-loops). **Hardware-wise, DynamiQ targets QPUs whose qubit connectivity is defined by a coupling graph $G_{hw}(Q_{phys}, R_{hw})$.** Per-edge two-qubit error rates and per-gate durations are optional refinements that default to constants when calibration data is unavailable. Further, DynamiQ preserves the inherited control structure: program gates never move across region boundaries and regions are never fused. Each region’s qubit footprint is therefore fixed by the source program. Transformations that restructure control-flow, such as fusing adjacent regions, are complementary and could be applied before DynamiQ runs. We also do not unroll loops, which can increase the precision of our analysis when the trip-count is known at compile-time. Fig. 4 shows an exemplar quantum circuit handled by our framework. Tab. 2 summarizes the notation used throughout this section.

Iterative Constructs (Loops). When DynamiQ encounters a loop, the loop body is extracted as an independent sub-circuit of the program and compiled separately (example in Fig. 3a). DynamiQ proceeds *recursively* on each new conditional or iterative construct.

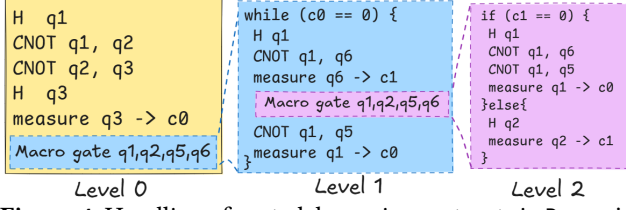


Figure 4. Handling of nested dynamic constructs in DynamiQ. Inner regions are compiled as independent blocks and abstracted as *macro gates* for outer routing.

Mapping loops with quantum operations demands knowing the mapping state φ_{in} at the loop entry in order to produce the sequence of SWAP operators S_{body} and the final state φ_{out} resulting from the mapping (updated from φ_{in}). However, to preserve the consistency of qubit mappings across loop iterations it is necessary to return to a known qubit state before proceeding to the next iteration. The traditional approach to this in the QISKIT compiler is to use S_{body}^{-1} (the same SWAPs applied in reverse order) to return to the known state φ_{in} .

Table 2. Definitions of symbols used in later sections.

Notation	Definition
q_i, p_j	Logical qubit i and physical qubit j .
$Q_{\text{logical}}, Q_{\text{phys}}$	Sets of logical and physical qubits.
φ	Mapping from Q_{logical} to Q_{phys} .
$G_{\text{hw}}(Q_{\text{phys}}, R_{\text{hw}})$	Hardware coupling graph.
R_{hw}	Directed edges of G_{hw} .
$D_{\text{phys}}[p_i, p_j]$	Shortest-path distance between p_i and p_j .
B	Dynamic block (loop or conditional branch).
ω_g	Dependence weight of gate g .
S	Sequence of SWAP gates.
S^{-1}	SWAP sequence obtained by reversing S , used to undo its effect.
C	Emitted gate sequence.

However, this strategy can be highly inefficient if the inherited mapping is of poor quality. To address this, DynamiQ applies a *bidirectional optimization process* to find a high-quality entry placement before fixing it (See Sec. 5.5).

Conditional Constructs. Branches of conditionals are extracted as independent sub-circuits fed to DynamiQ, recursively. (See Fig. 3b). Branches are optimized separately to generate valid hardware-aware mappings. The mapping obtained at the end of each branch represents the final placement $\varphi_{\text{out}}^{B_i}$ that would result if that specific control path were executed. Then, since all branches must eventually merge, DynamiQ evaluates all resulting branch mappings using the same cost model described in Sec. 5.4. Among these branches, the mapping with lowest overall cost is chosen as the *unified post-branch mapping*, denoted φ_{merge} . DynamiQ generates reconciliation passes for all branches to ensure convergence at the merge point. For each branch B_i , a minimal sequence of SWAP operations is computed—using the token-swapping method of Sec. 5.3—to transform the branch’s final mapping $\varphi_{\text{out}}^{B_i}$ into φ_{merge} . Each reconciliation sequence is appended to the end of its corresponding branch, guaranteeing that all branches terminate in the same qubit configuration.

As shown in Fig. 3b, this process introduces a reconciliation stage for all branches (*Branch 1* and *Branch 2*), to converge to a unified placement, later propagated forward for global consistency. This ensures deterministic routing and depth-efficient execution without requiring runtime evaluation of conditional paths.

Nested Dynamic Constructs. Nested dynamic control-flow regions are compiled independently with clearly defined entry and exit mappings, φ_{in} and φ_{out} . Because sub-circuits are reconciled to a known final placement, their exit configuration can be directly reused as the entry mapping of any enclosing or subsequent region. This design enables seamless integration of arbitrarily nested dynamic structures, such as loops within loops or conditionals inside iterative regions.

Fig. 4 shows an example nesting pattern in which dynamic blocks appear hierarchically, such as an if-else structure nested inside a while loop. At the outer level, the loop body (Level 1) includes an inner control-flow region (Level 2) that itself behaves as a dynamic block. Each level maintains its own entry and exit mappings, reconciled locally through the mechanism described in Sec. 5.3.

The compiler recursively invokes DynamiQ on a sub-block to determine the exit placement φ_{out}^B . Compilation proceeds as if B were a single static gate that transforms φ_{in}^B into φ_{out}^B .

5.2 IR Extraction

DynamiQ represents circuits with a hierarchical IR. At any level, individual gates with fixed qubit accesses and macro-gates (e.g., loops or conditionals) are modeled and used to capture transitive dependences. For each hierarchical level, we build the dependence relations R_{dep} over the gates visible at that level using the Integer Set Library [59]. We capture ordering constraints among gates, and then compute its transitive closure, $R_{\text{dep}}^+ = \text{TRANSITIVE_CLOSURE}(R_{\text{dep}})$, and derive the dependence weight of a gate g identified by t as: $\omega_g = \left| \{ t' \mid [t] \rightarrow [t'] \in R_{\text{dep}}^+ \} \right|$. Intuitively, ω_g measures how many gates (directly or indirectly) depend on g , capturing its global impact on the circuit. Gates with lower weights are less constrained by future computation and can be scheduled earlier with minimal disruption, while high-weight gates lie on many dependence paths and are more critical to the overall schedule [4]. Using the transitive closure ensures that ω_g reflects not only immediate dependences but all downstream effects, providing a global view that can be computed once and queried efficiently during analysis.

These abstractions form a robust foundation for analysis (Sec. 5.4) that enables us to overcome limitations of DAG-based methods on hierarchical, dynamic circuits.

5.3 Inter-Circuit Qubit Remapping: Qubit Reconciliation

This pass computes a minimal sequence of SWAP operations that transforms a given placement φ_{init} of one region into

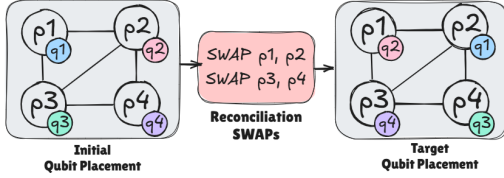


Figure 5. Token swapping between two qubit mappings, φ_{init} (left) and φ_{target} . Physical qubits p_i form the hardware graph, and logical qubits q_i are labeled tokens. A sequence of SWAPs transforms the initial mapping into the target.

another placement φ_{target} required by the next. Reconciliation is needed to revert back to a steady qubit state and for merging possible qubit states resulting from sub-circuits of disjoint control-flow paths. In practice, φ_{init} is the final mapping state of a loop body, φ_{body} , or the final state of a branch sub-circuit, $\varphi_{\text{out}}^{B_i}$.

Formal Problem Definition. Given $G = (Q_{\text{phys}}, R_{\text{hw}})$, this problem computes a sequence of remapping SWAPs S , that converts φ_{init} to φ_{target} .

This optimization problem is equivalent to the well-known *Token Swapping Problem* [6], where each logical qubit corresponds to a token initially placed on a vertex $\varphi_{\text{init}}(q)$ of the hardware graph G , and the objective is to move each token to its designated vertex $\varphi_{\text{target}}(q)$ using the fewest possible adjacent swaps along R_{hw} .

Example. Fig. 5 depicts the mapping of four qubits (q_i) to four physical locations, the p_i . Links $q_l \rightarrow q_k$ denote usable pairs of qubits. The logical-to-physical mappings before and after reconciliation are:

$$\varphi_{\text{init}} = \{q_1 \rightarrow p_1, q_2 \rightarrow p_2, q_3 \rightarrow p_3, q_4 \rightarrow p_4\},$$

$$\varphi_{\text{target}} = \{q_1 \rightarrow p_2, q_2 \rightarrow p_1, q_3 \rightarrow p_4, q_4 \rightarrow p_3\}.$$

The reconciliation sequence $S = [\text{swap}(p_1, p_2), \text{swap}(p_3, p_4)]$ transforms φ_{init} into φ_{target} , yielding a consistent mapping for the next circuit block.

Reconciliation Methodology. As the remapping problem is NP-hard, DynamiQ employs the 4-approximation algorithm proposed by Miltzow et al. [32]. This method iteratively selects SWAPs on adjacent qubits that reduce their aggregate distance to the target placement in the hardware connectivity graph, producing a sequence that closely approximates the minimal number of swaps. The algorithm runs in polynomial time and produces near-optimal reconciliation sequences in practice, making it well suited for integration within DynamiQ’s compilation pipeline.

Reconciliation enforces a target placement but does not choose it. The target depends on the type of dynamic block: for a loop it is the steady-state entry mapping, and for a conditional it is the mapping the branches merge into. How each is selected is described in Sec. 5.5.

5.4 Multi-Objective Mapping Cost Function

DynamiQ maps straight-line sub-circuits using a heuristic derived from QLOSURE [4] which prioritizes SWAPs that reduce

the dependency-weighted physical distance between dependent qubits. While effective for reducing routing overhead, this objective alone does not explicitly capture hardware error rates or the immediate impact of SWAPs on circuit depth and latency. To address this, DynamiQ augments this heuristic with two complementary terms: a *weighted-distance term* $C_{\text{spatial}}^{(s)}$, which incorporates error rates into the distance metric, and a *depth-rate term* $C_{\text{rate}}^{(s)}$, which penalizes SWAPs on qubits already heavily used in the current mapping.

Weighted-Distance Term ($C_{\text{spatial}}^{(s)}$). This term (Eq. 1) favors SWAPs that bring interacting qubits closer while still avoiding high-error routing paths. Candidate SWAPs are evaluated over a layered view of the circuit DAG (Fig. 6b), restricted to a lookahead window of L dependence layers. The window is built from the current front layer – the set of gates whose dependences have all been satisfied – by collecting gates in topological order of the DAG and grouping them into layers such that all gates within a layer are mutually independent. It terminates once L layers have been collected or the DAG is exhausted. For each circuit layer l , we define $\Gamma_l^{(s)}$ as the dependency-weighted average physical distance among interacting qubits after applying SWAP s [4]. Then, we scale up this distance by the average two-qubit gate error rate along the corresponding shortest path in the hardware graph (Fig. 6a), thereby explicitly incorporating hardware reliability into the cost.

$$C_{\text{spatial}}^{(s)} = \frac{\sum_{l=1}^L w_l \Gamma_l^{(s)}}{\sum_{l=1}^L w_l}, \quad w_l = \frac{1}{l}, \quad (1)$$

Above, each layer l is assigned a weight $w_l = 1/l$, so earlier layers (smaller l) contribute more to the score than later ones.

Depth-Rate Term ($C_{\text{rate}}^{(s)}$). Distance alone may still choose SWAPs that increase circuit depth. To reduce this effect, DynamiQ adds a penalty for using qubits that already appear frequently in the current schedule. In Fig. 6, after executing CNOT(q_4, q_5), swapping either participating qubit is more likely to increase the critical-path depth than using a less-active qubit. We capture this as

$$C_{\text{rate}}^{(s)} = \frac{\max(\text{depth}(\varphi_s(s_{q_1})), \text{depth}(\varphi_s(s_{q_2})))}{\max_{p_i \in Q_{\text{phys}}} \text{depth}(p_i)}, \quad (2)$$

where $\text{depth}(p)$ is the number of executed gate layers involving physical qubit p . A high $C_{\text{rate}}^{(s)}$ indicates that the SWAP uses qubits already on the critical path and is therefore penalized. This encourages the mapper to use other qubits when possible, improving parallelism and reducing latency.

In-Window Macro-nodes. These are built from the *current* control-flow level and region. Macro-nodes may contain control-flow regions and ordinary gates. Such a region contributes to $\Gamma_l^{(s)}$ in the same way any other node does: it is treated as a single multi-qubit gate over the union of accessed logical qubits (Sec. 4), so its pairwise qubit distances

enter the score directly, while its dependence weight ω_g , derived from the transitive closure R_{dep}^+ , already accounts for its downstream impact on the rest of the circuit. Its *internal* routing is not evaluated here: it is resolved by the recursive call under the region's own fixed $\varphi_{\text{in}}/\varphi_{\text{out}}$ mappings.

Composite Cost Function. Each candidate SWAP s is scored as

$$M(s) = \max(\delta_{s_{p_1}}, \delta_{s_{p_2}}) \left[\alpha C_{\text{spatial}}^{(s)} + \beta C_{\text{rate}}^{(s)} \right], \quad (3)$$

where δ is the standard decay factor used to discourage repeatedly swapping the same qubits [4, 28]. Parameters α and β control the trade-off between reducing routing overhead and limiting circuit depth. In summary, $C_{\text{spatial}}^{(s)}$ reduces communication cost and avoids high-error paths, while $C_{\text{rate}}^{(s)}$ helps reduce circuit depth and latency.

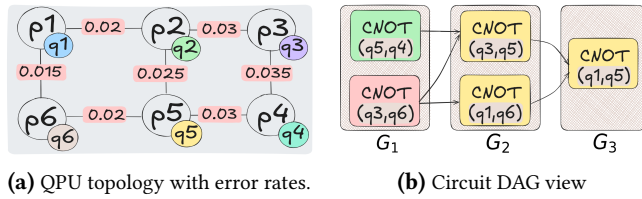


Figure 6. Illustration of the physical topology and the corresponding logical gate dependencies used in the latency-aware dependence-driven mapping example. Subfigure (a) shows the 6-qubit hardware graph with coupling errors, while subfigure (b) presents a logical circuit DAG.

5.5 Routing Algorithm

DynamiQ's mapping relies on three subcases (Alg. 1 for loops, Alg. 2 for entry-mapping optimization, and Alg. 3 for conditionals), and one driver algorithm, Alg. 4. Sub-circuits are extracted from the bodies of iterative and conditional constructs, converted to a dual DAG-based and polyhedral-based representation, and later used to perform the necessary mapping. To maintain mapping consistency, we assume that sub-circuits have single entry and exit points.

Algorithm 1 DynamiQ – Handling for/while Loops

Require: Loop node B with body C_{body} ; hardware map R_{hw} ; distance matrix D_{phys} ; current mapping φ
Ensure: Complete compiled block $C_{\text{entry}} \parallel C_{\text{loop}}$ and updated mapping φ_{in}^*

- 1: $\varphi_{\text{in}}^* \leftarrow \text{SELECTSTEADYSTATE}(G_{\text{body}}, R_{\text{hw}}, D_{\text{phys}}, \varphi)$ // Alg. 2
- 2: $S_{\text{entry}} \leftarrow \text{TOKENSWAP}(\varphi, \varphi_{\text{in}}^*, R_{\text{hw}})$ // Reconciliation SWAPs
- 3: $C_{\text{entry}} \leftarrow \text{EMITSWAPSEQ}(S_{\text{entry}})$; $\varphi \leftarrow \varphi_{\text{in}}^*$ // emits a sequence of SWAPs as IR
- 4: $[C_{\text{body}}, \varphi_{\text{body}}] \leftarrow \text{DYNAMIQ}(G_{\text{body}}, R_{\text{hw}}, D_{\text{phys}}, \varphi_{\text{in}}^*)$
- 5: $S_{\text{iter}} \leftarrow \text{TOKENSWAP}(\varphi_{\text{body}}, \varphi_{\text{in}}^*, R_{\text{hw}})$
- 6: $C_{\text{loop}} \leftarrow \text{EMITLOOP}(C_{\text{body}} \parallel \text{EMITSWAPSEQ}(S_{\text{iter}}))$
- 7: **return** $(C_{\text{entry}} \parallel C_{\text{loop}}, \varphi_{\text{in}}^*)$

Mapping Iterative Constructs. Alg. 1 is used by DynamiQ to map iterative control-flow structures, for-loops and while-loops. First, a *steady-state* qubit mapping φ_{in}^* is computed by the SELECTSTEADYSTATE (Alg. 2), which identifies a mapping that remains stable across loop iterations. Once the steady-state is decided, we invoke the TOKENSWAP algorithm [32] to determine the sequence of SWAP gates that convert

Algorithm 2 SelectSteadyState: bidirectional passes

Require: Loop body DAG G_{body} , hardware map R_{hw} , distance matrix D_{phys} , current mapping φ , number of passes $K \geq 1$
Ensure: Optimized loop entry mapping φ_{in}^*

- 1: $\varphi_{\text{seed}} \leftarrow \varphi$
- 2: **for** $k = 1$ to K **do**
- 3: $[_, \varphi_{\text{fwd_out}}] \leftarrow \text{DYNAMIQ}(G_{\text{body}}, R_{\text{hw}}, D_{\text{phys}}, \varphi_{\text{seed}})$
- 4: $G_{\text{rev}} \leftarrow \text{REVERSEDAG}(G_{\text{body}})$ // invert edges and two-qubit direction
- 5: $[_, \varphi_{\text{bwd_out}}] \leftarrow \text{DYNAMIQ}(G_{\text{rev}}, R_{\text{hw}}, D_{\text{phys}}, \varphi_{\text{fwd_out}})$
- 6: $\varphi_{\text{seed}} \leftarrow \varphi_{\text{bwd_out}}$ // seed next round
- 7: **end for**
- 8: $\varphi_{\text{in}}^* \leftarrow \varphi_{\text{seed}}$; **return** φ_{in}^* // final mapping after K forward/backward passes

the initial mapping state, φ_{in} —i.e., the mapping at the point just before entering the loop—into φ_{in}^* . The found sequence of SWAP gates, S_{entry} , is emitted as a sequence of SWAP operations and prepended to the loop body circuit C_{body} (i.e., the compiled representation of the loop body), after which the current mapping state is updated to φ_{in}^* .

As illustrated in Fig. 7, DynamiQ then takes the loop body B_{body} and maps individual sub-circuits, those in disjoint control-flow paths, in a recursive manner, returning a mapped circuit body C_{body} and the outgoing qubit state from it, φ_{out} . The latter is reconciled back to φ_{in}^* via TOKENSWAP, generating a sequence S_{iter} that ensures consistency across iterations. Finally, these reconciliation SWAPs are appended to the loop body, forming a self-consistent compiled loop.

As having input/output mappings that require few SWAPs is necessary, we use Alg. 2 to search for such mapping states via a short-fixed number of passes K , using bidirectional passes: alternating forward and backward executions of the mapping algorithm over the circuit and its reverse, repeated K times, typically converging after 3–4 passes. A similar approach has been previously used in [28] to find initial logical-to-physical qubit mappings, using $K = 3$.

Algorithm 3 DynamiQ – Handling if-else Conditionals

Require: Conditional node B with branch subgraphs $\{G_i\}$; hardware map R_{hw} ; distance matrix D_{phys} ; entry mapping φ
Ensure: reconciled conditional C_{cond} and updated mapping φ

- 1: **for each** branch i **do**
- 2: $[C_i, \varphi_{\text{out}}^{B_i}] \leftarrow \text{DYNAMIQ}(G_i, R_{\text{hw}}, D_{\text{phys}}, \varphi)$
- 3: **end for**
- 4: $\varphi_{\text{merge}} \leftarrow \text{CHOOSEUNIFIEDMAPPING}(\{\varphi_{\text{out}}^{B_i}\})$; $C_{\text{cond}} \leftarrow \text{BEGINIFELSE}()$
- 5: **for each** branch i **do**
- 6: $S_i \leftarrow \text{TOKENSWAP}(\varphi_{\text{out}}^{B_i}, \varphi_{\text{merge}}, R_{\text{hw}})$
- 7: $C_{\text{cond}} \leftarrow C_{\text{cond}} \parallel \text{EMITBRANCH}(C_i \parallel \text{EMITSWAPSEQ}(S_i))$
- 8: **end for**
- 9: $C_{\text{cond}} \leftarrow \text{ENDIFELSE}(C_{\text{cond}})$; $\varphi \leftarrow \varphi_{\text{merge}}$; **return** $(C_{\text{cond}}, \varphi)$

Mapping Conditional Constructs. Alg. 3 presents the steps to map sub-circuits in disjoint control-flow paths (lines 1–3). Each sub-circuit G_i in each path i is mapped with Alg. 4, **Then, the resulting branch with the lowest cost is selected as φ_{merge} . After this, we find and append reconciliation passes that convert each $\varphi_{\text{out}}^{B_i}$ to φ_{merge} —yielding one exit mapping $\varphi_{\text{out}}^{B_i}$ per branch. CHOOSEUNIFIEDMAPPING (Alg. 3, line 4) then merges these into a single placement: it scores each $\varphi_{\text{out}}^{B_i}$ with C_{spatial} (Eq. 1), evaluated over the lookahead window built from the front layer immediately after the merge**

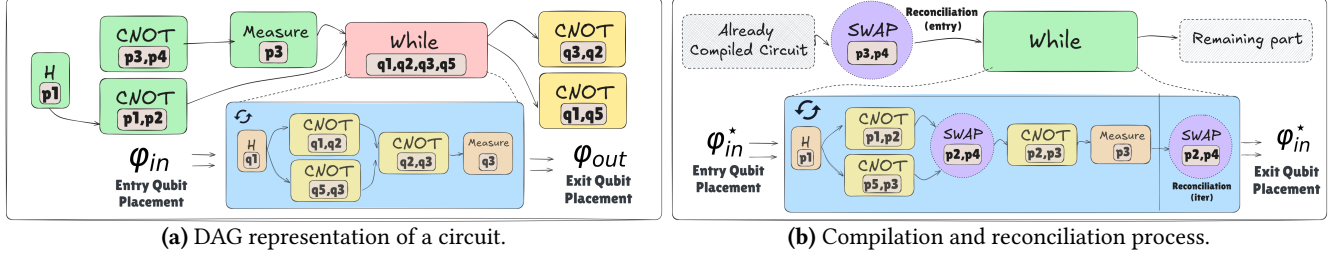


Figure 7. Illustration of the loop compilation process performed by DynamiQ. Subfigure (a) shows the DAG-level structure of the circuit with qubit placements ϕ_{in} and ϕ_{out} . Subfigure (b) depicts the compilation flow integrating two reconciliation passes: one at loop entry (S_{entry}) to align the qubit layout with the steady-state mapping ϕ_{in}^* , and one per iteration (S_{iter}) to restore this mapping after each iteration. These transformations respect the device connectivity constraints shown in Fig. 6a.

point, and keeps the lowest-scoring mapping as ϕ_{merge} . The score therefore weighs downstream routing overhead, gate criticality through ω_g , and hardware reliability through the per-edge error rates.

Branch Probabilities. DynamiQ weights all branches with equal probability, which is the default when the branch distribution is unknown at compile-time. When probabilities p_i are available, they can be folded in by scaling each score by $(1 - p_i)$. All experiments assume uniform probabilities.

Putting It All Together. Finally, we present the driver algorithm used in DynamiQ. The algorithm first performs initialization tasks (lines 1–4) that create an empty circuit, prepare the circuit front layer (with schedulable gates), compute the transitive closure among gates [4], and start qubit decay factors (which avoid reusing the same qubit in mapping paths for a short time frame). The main loop (lines 5–26) handles the three distinct cases of a circuit: i) when executable gates are ready (lines 7–9), they are directly emitted and the front layer is advanced; ii) when the front layer contains only control-flow constructs (lines 10–16), a block is selected (line 11) and handled via Alg. 1 or Alg. 3 (lines 12–16), after which the compiled block is appended (line 17) and spliced out of the DAG (line 18); iii) otherwise, SWAP candidates are generated, evaluated using the cost model, and the best SWAP is applied to update the mapping and circuit (lines 21–24).

Once a sub-circuit B exhibiting control-flow is mapped, B is spliced out of G to prepare for subsequent gates.

Example (Fig. 7): Mapping starts by fixing connectivity for the static gates, adding and scheduling the necessary SWAPs where required. In Fig. 7(a), the gates shown in green can all be executed directly because their corresponding physical qubits are adjacent on the target hardware topology (Fig. 6a). Next, we encounter a dynamic block—a while-loop—and thus invoke Alg. 1. We begin by identifying the steady-state qubit mapping ϕ_{in}^* using Alg. 2 on the body of the loop. To reconcile the current mapping with this steady state, an additional SWAP between p_3 and p_4 is inserted before entering the loop, as shown in Fig. 7b. This operation tries to resolve

Algorithm 4 DynamiQ: Recursive Routing over Dynamic Blocks

Require: Outer DAG G (gates, measurements, dynamic blocks), hardware map R_{hw} , distance matrix D_{phys} , initial mapping ϕ
Ensure: Routed circuit C and final mapping ϕ

```

1:  $C \leftarrow \epsilon$  // empty output circuit
2:  $L_f \leftarrow \text{INITFRONTLAYER}(G)$ 
3:  $R^+ \leftarrow \text{TRANSITIVECLOSURETWOQ}(G)$  // for scoring only
4:  $\delta \leftarrow 1$  on  $Q_{phys}$  // decay/usage vector
5: while  $L_f \neq \emptyset$  do
6:    $G_{ready} \leftarrow \text{EXTRACTREADYGATES}(L_f, \phi, R_{hw})$ 
7:   if  $G_{ready} \neq \emptyset$  then
8:      $C \leftarrow C \parallel \text{EMITGATES}(G_{ready})$ 
9:      $\text{ADVANCE}(G_{ready}, G, L_f)$ ;  $\delta \leftarrow 1$ 
10:  else if  $\text{HASONLYCONTROLFLOW}(L_f)$  then // for/while or if-else
11:     $B \leftarrow \text{PICKCONTROLNODE}(L_f)$ 
12:    if  $\text{type}(B) \in \{\text{for}, \text{while}\}$  then
13:       $[C_B, \phi] \leftarrow \text{HANDLELOOP}(B, R_{hw}, D_{phys}, \phi)$  // Alg. 1
14:    else
15:       $[C_B, \phi] \leftarrow \text{HANDLECONDITIONAL}(B, R_{hw}, D_{phys}, \phi)$  // Alg. 3
16:    end if
17:     $C \leftarrow C \parallel C_B$ 
18:     $\text{SPICECOMPILEDBLOCKANDADVANCE}(G, B, L_f)$  // update  $L_f$ 
19:  else
20:     $L_w \leftarrow \text{MAKELOOKAHEAD}(L_f, G)$ 
21:     $\mathcal{G} \leftarrow \text{BUILDLOOKAHEADLAYERS}(L_w, G)$ 
22:     $S \leftarrow \text{GENSWAPCANDIDATES}(L_f, R_{hw})$ 
23:     $s^* \leftarrow \arg \min_{s \in S} M(s, R^+, \mathcal{G}, D_{phys}, \delta)$  // Sec. 5.4
24:     $\phi \leftarrow \phi \circ s^*$ ;  $C \leftarrow C \parallel \text{EMITSWAP}(s^*)$ ;  $\delta \leftarrow \text{UPDATEDECAY}(\delta, s^*)$ 
25:  end if
26: end while
27: return  $(C, \phi)$ 

```

certain disconnections inside the loop before actually entering it. Next, we recursively invoke DynamiQ on the loop body, where another SWAP between p_2 and p_4 is introduced to enable execution of the CNOT(q_2, q_3) gate. Because this SWAP changes ϕ_{in}^* , a final reconciliation pass is added at the end of the loop to restore ϕ_{in}^* for the next iteration. Once the loop finishes execution, compilation proceeds with the remaining portion of the circuit.

6 Dynamic-QUEKO: Hierarchical Circuits

We build circuits from a simple grammar over node types: $N ::= \text{leaf} \mid \text{seq}(N^+) \mid \text{for}(K, N) \mid \text{while}(q, N) \mid \text{ifelse}(q, N, N)$.

A leaf is a static block with depth leaf_depth and one- and two-qubit gate densities (δ_1, δ_2) where (δ_1, δ_2) specify the fraction of qubits active per layer. seq composes sub-circuits sequentially, $\text{for}(K, N)$ is a counting-loop with K iterations, $\text{while}(q, N)$ is a measurement-controlled loop

and finally $\text{ifelse}(q, \mathcal{N}, \mathcal{N})$ defines a conditional branch controlled by a measurement outcome associated with qubit q . This grammar generates nested, parameterized circuits. Examples of generated circuits are included in the supplemental material.

d-QUEKO circuits generate an underlying topology for a given number of physical qubits, `n_qubits`. Edges are added with probability $\rho \in [0, 1]$. This grants control over routing difficulty: ρ sets the connectivity density (higher ρ increases parallelism). A density of $\rho = 1$ corresponds to a complete graph. $\rho \times (|V| - 1)$ is the average number of links for any given qubit. `leaf_depth` sets the depth of each static region and is the main knob for complexity.

7 Evaluation

7.1 Experimental Testbed

We validate the efficiency of DynamiQ against QISKIT 1.3.2 and its default routing pass, LightSABRE [68], the only mapper that supports circuits with dynamic control-flow. LightSABRE is invoked with `routing_method='sabre'` and `layout_method='identity'`, `optimization_level=1`, all remaining options at their defaults, and `seed_transpiler` swept over 10 seeds. We select this setup to preserve the gate count and the initial placement across both mappers, and focus on the impact of control-flow in the mapping process. For the IBM QPUs, the 127-qubit Brisbane and the 156-qubit Kingston, we use calibration data collected on October 17, 2025. Experiments aim to demonstrate the quality of our mapper across four metrics: circuit depth, SWAP count, circuit latency, and incurred error along the critical path.

This evaluation consists of five experiments: a) Benchmarks on d-QUEKO (Sec. 7.2), b) a comparison on chiplet architectures (Sec. 7.3), c) an experiment using surface codes (Sec. 7.4), d) scalability results (Sec. 7.5), and e) ablation studies (Sec. 7.6).

Evaluated Circuits. We use dynamic circuits from d-QUEKO, Stim [19], and static circuits from QUEKO [56]. d-QUEKO exposes data-dependent control flow with controlled coverage of the structural dimensions that define mapping difficulty: nesting depth and control-flow shape (grammar $\mathcal{N}$), static-region size (`leaf_depth`, swept 10–90), circuit width (54/81/121/256 qubits), and gate density (δ_1, δ_2). With Stim we generate rotated surface-code circuits (Sec. 7.4) with standard, externally-defined QEC workloads: memory-Z experiments swept over code distances $d \in \{3, 5, 7\}$ and rounds $r \in \{3, 5, 10, 15, 20\}$, providing an independent check on d-QUEKO results in the regime of adaptive quantum circuits.

Performance Metrics. We use four metrics to compare quantum mappers: **SWAP count** measures the total number of SWAP operations in the trace; **Circuit depth** measures the minimal number of parallel layers under qubit exclusivity; **Latency** accumulates per-qubit gate durations from

hardware calibration data, while **Error** accumulates per-gate physical error rates on each qubit and averages them to obtain the overall error of the compiled circuit, similar to prior work [51]. These metrics are extracted from the CP (Critical Path) of mapped circuits, except for the SWAP count, which considers the total number of SWAP gates introduced by each mapper. We assume that all loops execute for ten iterations. For conditional blocks, all branches are evaluated and the branch with the worst-case result is selected for every metric. The same methodology is applied, recursively, across nested constructs. The improvements ($\Delta\%$) reported in the tables are computed for each metric as $(\text{Val}_{\text{baseline}} - \text{Val}_{\text{ours}}) / \text{Val}_{\text{baseline}} \times 100$.

Research Questions. Our evaluation aims to answer the following research questions: RQ1) Is DynamiQ more effective than baseline methods? RQ2) Are qubit reconciliation, loop entry remapping, and the multi-objective function the underlying reasons for the improvement in dynamic circuits?

7.2 Evaluating DynamiQ on the d-QUEKO Benchmark

This section studies the impact of DynamiQ on d-QUEKO dynamic circuits. We generate benchmarks of the form:

$\mathcal{N} = \text{seq}((\text{leaf}, \text{while}(q, \text{leaf}), \text{leaf})^{\times 10})$, i.e., the three-block pattern is placed *ten times* in a row. For each such structure we generate 10 distinct circuits, and we do this for circuits using 54, 81, and 121 qubits, while varying leaf depths from 10 to 90. Later, in Sec. 7.6.1, we extend the exploration to deeper nested structures defined by $\mathcal{N}$.

Table 3. Average relative improvements (%) of DynamiQ over Qiskit Sabre across two IBM backends (Higher is better). Benchmark names are abbreviated as dQ- n , where n is the qubit count. Metric abbreviations are: S = SWAPs, D = Depth, L = Latency, and E = Error.

Bench.	Brisbane (127 qbt)				Kingston (156 qbt)			
	S	D	L	E	S	D	L	E
dQ-54	52.04	13.92	13.06	14.62	39.46	11.48	11.16	40.53
dQ-81	40.97	14.66	13.99	21.08	48.71	18.48	18.35	38.99
dQ-121	51.73	17.75	18.59	16.87	34.54	8.70	8.80	33.62

The summary of our results is shown in Tab. 3 while Fig. 8 shows the absolute values for IBM Brisbane. We compare with QISKIT Sabre, given that it is the only one that supports dynamic circuits. We first turn our attention to the high-level summaries in Tab. 3. We observe a marked improvement across both back-ends, Brisbane (127 qubits) and Kingston (156 qubits). The most notable improvements are achieved on the SWAP and Error metrics.

Improvements are owed to the balance between *weighted distance* (C_{spatial}) and *depth-rate* (C_{rate}) terms (Sec. 5.4). The former term avoids swap insertion across the critical path while also avoiding noisy qubits and high-error links. Further, effects are compounded due to the *reconciliation pass* (Sec. 5.3) and *entry re-mapping*. In regard to the circuit Depth

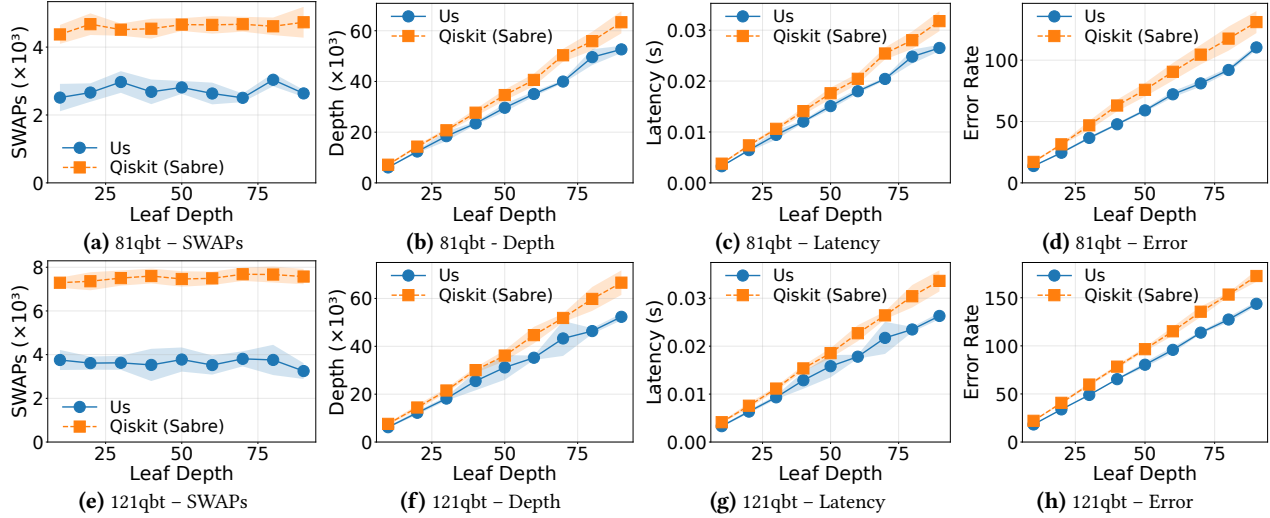


Figure 8. Routing performance on **IBM Brisbane** over **10 random seeds** using the **Dynamic QUEKO benchmarks**. Plots show the mean with $\pm 1\sigma$ shaded (our method vs. Qiskit Sabre) across leaf depth. Figures (a)–(d) and (e)–(h) correspond to circuit widths, across the four metrics. X-axis (Leaf Depth) is the circuit depth of the static circuit in the innermost nested loop.

and Latency metrics, DynamIQ improves between 8.7% and 18.59% over Sabre. Overall, our results translate to faster circuits that make use of 34%–52% fewer SWAP gates, and between 14.62%–40.53% lower error.

7.3 Evaluation on Chiplet-Based QPUs

We evaluate DYNAMIQ on chiplet-based QPUs using generated d -QUEKO circuits, including a 256-qubit benchmark that stresses inter-chip communication (Table 4). On hexagon heavy $8 \times 8_{2 \times 2}$ topologies (four chips in a 2×2 grid, generated using MECH [67]), DYNAMIQ consistently improves all metrics over Qiskit Sabre, reducing SWAP count by 34–38%, depth by 4–13%, latency by 14–16%, and error by 10–12%. On IBM Flamingo (three 156-qubit chips with a $7.4 \times$ inter-chip penalty), DYNAMIQ deliberately increases SWAPS (+32%) and depth (+20%) to avoid costly inter-chip communication. This trade-off reduces latency by 2.8% and error by 15%, demonstrating effective hardware-aware optimization.

Table 4. Routing results on chiplet-based QPUs: Heavy-Hexagon (Heavy H.) and IBM Flamingo (IBM F.) using 81, 121 and 256 d -QUEKO circuits. Improvement denoted with +P%, degradation denoted with –P%.

Topology	Qubits	SWAPs ($\times 10^3$)		Depth ($\times 10^3$)		Latency ($\times 10^3 \mu s$)		Error	
		Ours	Sabre	Ours	Sabre	Ours	Sabre	Ours	Sabre
Heavy H.	81	2.97	4.51	30.1	34.8	45.8	54.6	11.38	12.88
Heavy H.	121	4.57	7.41	33.0	34.4	50.4	58.3	12.31	13.65
Overall imp.		+36.2%		+8.7%		+14.9%		+10.7%	
IBM F.	256	15.9	12.1	40.9	34.2	5122.0	5270.4	154.40	181.72
Overall imp.		–31.8%		–19.6%		+2.8%		+15.0%	

7.4 Evaluating on Rotated Surface-Code Circuits

We evaluate DynamIQ on rotated surface-code circuits generated using STIM [19]. Each circuit implements a memory-Z experiment using $2d^2 - 1$ qubits (d^2 data + $d^2 - 1$ ancilla) with a four-step cx schedule. We consider two backends:

IBM Brisbane (127 qubits, heavy-hex) and a chiplet-based MECH [67] topology (480 qubits, 3×4 array). For Brisbane, we sweep $d \in \{3, 5, 7\}$ and $r \in \{3, 5, 10, 15, 20\}$, where d denotes the code distance and r the number of syndrome extraction rounds, with three replicates and three routing seeds (135 runs per method). For MECH, we use $r=3$ with the same replicates and seeds (27 runs).

Table 5 reports per-distance averages. On IBM Brisbane, DYNAMIQ reduces SWAP count by 31.5%, depth by 45.7%, latency by 48.1%, and error by 28.4% on average. On the MECH chiplet topology, it achieves similar gains, reducing SWAPs by 29.6%, depth by 27.0%, latency by 42.1%, and error by 28.6%. Overall, DYNAMIQ consistently improves all metrics across both backends. On chiplets, it further adapts routing to minimize costly cross-chip interactions, leading to particularly strong latency and error reductions despite limited inter-chip connectivity.

Table 5. Performance metrics of DynamIQ vs. Qiskit Sabre on rotated surface-code circuits across backends. Summary row, Overall imp., shows relative (percentual) improvement with +P% indicating improvements.

Backend	d	Qubits	CX/rnd	SWAPs		Depth		Latency		Error	
				Ours	Sabre	Ours	Sabre	Ours	Sabre	Ours	Sabre
Brisbane	3	17	24	848	1360	395	562	229	355	1.92	2.84
	5	49	120	4320	4887	806	1132	494	711	3.80	4.47
	7	97	168	8271	13387	909	2192	564	1415	5.05	7.71
Overall imp. $\Delta\%$				+31.5%		+45.7%		+48.1%		+28.4%	
MECH	3	17	24	420	480	200	185	124	112	0.09	0.10
	5	49	120	1462	2380	316	446	472	737	0.17	0.27
	7	97	168	4552	6284	460	706	526	1091	0.25	0.33
Overall imp. $\Delta\%$				+29.6%		+27.0%		+42.1%		+28.6%	

7.5 Scalability of DynamIQ Mapping

Tab. 6 presents the average mapping time of DynamIQ on d -QUEKO dynamic circuits with 54, 81, and 121 qubits, across

Table 6. Average mapping time (s) for DynamiQ on **dynamic circuits** across three leaf-depth categories: Small (10–30), Medium (40–60), and Large (70–90). Leaf Depth is the circuit depth of the static circuit in the innermost nested loop.

Qubits	IBM Kingston (156 Q)			IBM Brisbane (127 Q)		
	Small	Med.	Large	Small	Med.	Large
54	0.22	0.48	0.73	0.19	0.37	0.56
81	0.38	0.69	1.01	0.34	0.64	0.97
121	0.86	1.48	2.03	0.67	1.10	1.52

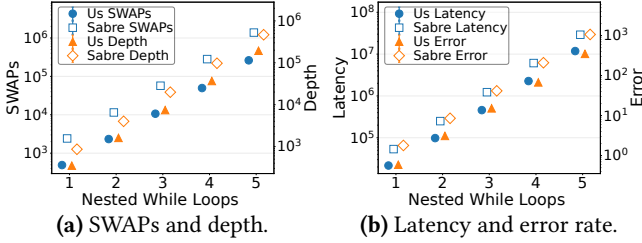


Figure 9. Dynamic nested-loop circuits on 81-qubit circuits, *ibm_kingston*. Four metrics shown on left and right Y-axes averaged over 3 random seeds and reported as a function of the number of nested while loops. Error bars omitted because of close proximity among values. Lower is better.

varying leaf-depth categories. **All mapping times were measured on a single core of an Intel Core i7-10750H (2.60 GHz) with 8 GB of RAM.** In general, we observe a scalable mapping time where the number of back-end qubits exerts a first order effect, while the qubit footprint of the benchmark set increases the time in a linear fashion. Additional results on the mapping time for the IBM Kingston and IBM Brisbane backends are in the supplemental material.

Table 7. Average percentage improvement (in %) over the reconciliation-only baseline on dynamic circuits. Configurations are cumulative: (i) Recon, (ii) Recon+Err, (iii) Recon+Err+Depth, and (iv) Recon+Err+Depth+Remap. Metrics are SWAP count (S), depth (D), latency (L), and error (E). Positive values indicate better performance.

Method	81 Qubits				121 Qubits			
	S	D	L	E	S	D	L	E
(ii) Recon+Err	-6.9	-2.3	-2.2	15.1	-0.3	2.1	3.5	13.5
(iii) Recon+Err+Depth	17.7	6.5	6.1	9.7	17.5	6.6	6.9	10.0
(iv) Recon+Err+Depth+Remap	48.2	17.7	17.1	20.5	55.5	19.4	20.1	18.0

7.6 Ablation Studies

7.6.1 Impact of Loop Depth in Mapping Efficiency. We first explore the performance of DynamiQ on circuits with diverse hierarchical structure. Fig. 9 shows five classes of circuits with 1-5 nested loops, each with 1-5 if-conditionals nested in them. (Examples of circuits included in the supplemental material). Metrics are computed assuming that all loops execute the full trip-count (10 iterations). For conditional structures, we adopt a worst-case analysis among all of the branches. As shown in Fig. 9, DynamiQ achieves

significant gains across evaluated metrics, with the circuit complexity primarily increasing with the depth of while loops. On average, we observe an **80% reduction in SWAP count** and **50–60% improvement** in latency, error, and circuit depth compared to Qiskit’s SABRE routing algorithm.

7.6.2 What Drives Improvement on Dynamic Circuits.

To attribute the contributions of individual mapping components, we perform an ablation study. Components are added one by one. Tab. 7 reports the average percentage improvement of each configuration over the baseline, and Fig. 10 shows the corresponding trends across metrics. The baseline, *Recon* (i), uses only the reconciliation passes to preserve mapping consistency across dynamic regions. It does not use the error-aware term in the cost function, does not include the depth-rate term C_{rate} , and does not optimize loop-entry mappings through steady-state selection. We then add components cumulatively: *Recon+Err* (ii) adds hardware-error awareness to the cost function; *Recon+Err+Depth* (iii) further adds the depth-rate term C_{rate} ; and *Recon+Err+Depth+Remap* (iv) additionally enables steady-state loop-entry remapping through `SELECTSTEADYSTATE`. Thus, configuration (iv) corresponds to the full DynamiQ algorithm.

Tab. 7 reports improvements for four metrics: SWAP count (S), circuit depth (D), latency (L), and error (E). Positive values indicate improvement over the reconciliation-only baseline. The results show that adding error awareness primarily improves E, adding C_{rate} improves D and L, and adding steady-state remapping yields the largest overall gains by reducing repeated routing overhead inside loops.

Table 8. Sensitivity to $\langle \alpha, \beta \rangle$ on heavy-hexagon 8x8-2x2 QPU, queko-bss-54qbt. Baseline is *spatial_only* $\langle \alpha, \beta \rangle = \langle 1, 0 \rangle$. Improvement is computed as $(\text{Baseline} - \text{Variant}) / \text{Baseline} \times 100$ (positive is better). Pair $\langle 1.0, 0.2 \rangle$ (bottom row) is used in the evaluation.

α	β	SWAPs (%)	Depth (%)	Latency (%)	Error (%)
0.50	0.50	-42.00	8.00	-23.83	-23.25
0.60	0.40	-21.98	9.82	-9.73	-10.63
0.80	0.20	-11.47	11.35	-1.82	-5.35
1.00	0.40	-11.19	16.99	2.11	-4.06
1.00	0.20	-2.61	12.40	6.04	0.35

7.6.3 Ablation Study of the Cost Model Parameters.

We evaluate the sensitivity of DYNAMIQ’s cost model to the (α, β) parameters (Eq. 3), which balance communication (C_{spatial}) and depth (C_{rate}) objectives. Tab. 8 shows a shallow sweep of $\langle \alpha, \beta \rangle$ pairs; it highlights the impact relative to the spatial-only mode, $\langle \alpha, \beta \rangle = \langle 1, 0 \rangle$. We observe that pairs with $\alpha < 1$ consistently degrade performance while larger α prioritize reducing communication-driven costs (distance/error). Increasing β improves depth, but at the expense of higher SWAP count and latency, reflecting a trade-off between short-term scheduling and long-term communication overhead. (NOTE: All previous experiments use $\langle \alpha, \beta \rangle = \langle 1.0, 0.2 \rangle$)

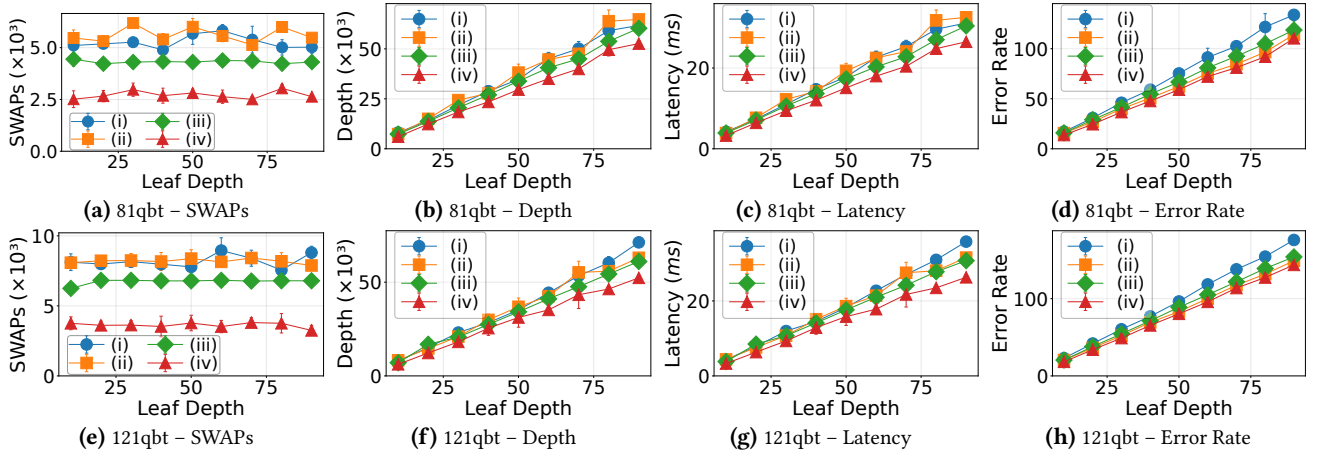


Figure 10. Ablation study on BRISBANE back-end. Each panel shows mean values with $\pm 1\sigma$ error bars across leaf depths over **10 random seeds**. Four levels are considered: (i) **Recon** (reconciliation only), (ii) **Recon+Err** (reconciliation with error awareness in the cost function), (iii) **Recon+Err+Depth** (error-aware reconciliation with additional depth regularization), and (iv) **Full method** (the complete DynamiQ model with all components enabled). Figures (a)–(d) and (e)–(h) correspond to circuit widths, across the four metrics. X-axis (Leaf Depth) is the circuit depth of the static circuit in the innermost nested loop.

7.7 Evaluation on Static Circuits

~~Due to space limitations and to the main subject of this work, we defer results on static circuits to supplemental material.~~

8 Related Work

Dynamic Circuits. Niu et al. introduce a suite of synthesis techniques for dynamic quantum circuits, integrated into the BQSKit framework [36, 37]. Their approach leverages unitary decomposition techniques and topology-specific numerical synthesis to prepare arbitrary states prior to the qubit mapping passes. On the other hand, QISKIT relies on specialized control-flow constructs that simplify capturing quantum gates as sub-circuits, which are later transpiled. Chen et al. propose *Sword* [10], which segments circuits into a main and controlled sub-circuits, each mapped using qubit occupancy and regional gate density. Wall-clock time is used as objective, and each sub-circuit is assumed to execute once. Sang et al. [49] allocate qubits to cores in modular architectures with a learned policy, using mid-circuit measurement for qubit reuse rather than control-flow, so the circuit structure remains static. Cubeddu et al. [14] address unstructured control-flow, weighting basic blocks by expected execution count and restoring placement consistency with inverse SWAPs placed according to the dominator relation [20]. Their goal of a unique placement matches ours, although we obtain it by selecting a steady-state entry mapping per region rather than undoing each SWAP. Ryan et al. [48] describe the control electronics behind dynamic execution, confirming that the constructs we target are realizable within qubit coherence times. TeLSABRE [47] extends SABRE with qubit and gate teleportation for inter-core movement alongside intra-core SWAPs. They harness new

primitives to support routing static DAGs, whereas DynamiQ consumes the program structure. QGo [65] splits mapped circuits into small blocks to re-synthesize them and reduce CNOT count. Lourens et al. [29] build Quantum CNN architectures by nesting reusable circuit blocks (convolution and pooling layers) across several levels, evidence that quantum circuits are naturally written hierarchically. Cheng et al. [11] perform qubit mapping on 2D-grids by reducing a weighted Manhattan distance frequency of qubit pair interactions. It then inserts SWAPs using a priority queue. Annechini et al. [2] scores SWAPs by their scheduling depth rather than by path length, cutting routed depth by up to 70%. but targets routing of purely static circuits.

Affine-Based Mappers. Qlosure [4] introduces a dependence driven method for scalable qubit mapping in static circuits based on the transitive closure of dependence relations. Crucially, it only supported static circuits and driven solely by the circuit depth objective and inter-qubit distance. In contrast, our work targets dynamic, hierarchical circuits across error, latency, SWAP count, and circuit depth.

Qubit Reuse [16, 24, 25]. This method involves resetting a qubit once its quantum state is collapsed by measurement. After this, the qubit can be recycled to avoid using new ancilla qubits, thereby reducing the circuit width. Nonetheless, qubit reuse is orthogonal to the problem of qubit mapping in both static and dynamic circuits. QRCC [41] analyzes qubit lifetimes using a qubit-reuse-aware DAG and formulates qubit reuse as an integer linear program (ILP), while Jiang et al. [26] use a qubit dependency graph (QDG) to analyze and validate recycling strategies. QR-Map [27] uses map-based abstractions to identify reuse opportunities. Qulin [25] transforms conditional multi-qubit gates to a linear number of

two-qubit operations without helper qubits, reducing code size while preserving correctness.

Uncomputation. Unqomp [39] automates uncomputation in circuits by returning helper qubits to the $|0\rangle$ state after use. SQUARE [16] optimizes the allocation and reclamation of ancilla qubits through uncomputation, balancing trade-offs among qubit, gate count, and parallelism. Venev et al. [58] leverage a custom, modular IR to synthesize uncomputation and adjoints.

9 Conclusion

This paper introduces DynamiQ, a new qubit mapping framework for dynamic circuits. DynamiQ effectively maps hierarchical quantum circuits exhibiting data-dependent control-flow, which gives rise to multiple sub-circuits. We make four technical contributions. First, we leverage the key concept of *qubit reconciliation*, a pass that unifies the qubit mapping state from different control-flow paths, thereby enabling consistent mappings across sub-circuit and hierarchical boundaries. Second, we introduce a novel block-entry remapping that optimizes the qubit steady-state to which all control-flow paths must merge. Third, we enhance a multi-objective cost function based on transitive dependences, qubit depth-rate and qubit error that allows us to optimize all four circuit metrics. Finally, we present the design and implementation of d-QUEKO (Dynamic QUEKO), a generator of dynamic circuits exhibiting varying control structure. We validate our approach on a wide range of dynamic circuits on two monolithic QPUs of 127 and 156 qubits, and on chiplet hexagon-based QPUs. On the former, DynamiQ improves the SWAP count by up to 52%, depth by up to 18%, latency by up to 18.6%, and error by up to 40%. On chiplet architectures, we achieve improvements of up to 36% on SWAP count, 8.7% on depth, 15% on latency, and 15% of error.

References

- [1] Google Quantum AI. 2025. Cirq: An open source framework for programming quantum computers. <https://quantumai.google/cirq>
- [2] Alessandro Annechini, Marco Venere, Donatella Sciuto, and Marco D. Santambrogio. 2025. DDRoute: a Novel Depth-Driven Approach to the Qubit Routing Problem. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. 1–7. doi:10.1109/DAC63849.2025.11133018
- [3] Juan Miguel Arrazola, Olivia Di Matteo, Nicolás Quesada, Soran Jangiri, Alain Delgado, and Nathan Killoran. 2022. Universal quantum circuits for quantum chemistry. *Quantum* 6 (June 2022), 742. doi:10.22331/q-2022-06-20-742
- [4] Marouane Benbetka, Merwan Bekkar, Riyadh Baghdadi, and Martin Kong. 2025. Dependence-Driven, Scalable Quantum Circuit Mapping with Affine Abstractions. arXiv:2510.27067 [cs.PL] <https://arxiv.org/abs/2510.27067>
- [5] Debjyoti Bhattacharjee, Abdullah Ash Saki, Mahabubul Alam, Anupam Chattopadhyay, and Swaroop Ghosh. 2019. MUQUT: Multi-constraint quantum circuit mapping on NISQ computers. In *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, IEEE, Westminster, Colorado, USA, 1–7.
- [6] Édouard Bonnet, Tillmann Miltzow, and Paweł Rządewski. 2018. Complexity of token swapping and its variants. *Algorithmica* 80, 9 (2018), 2656–2682.
- [7] Natalie C. Brown, John Peter Campora III, Cassandra Granade, Bettina Heim, Stefan Wernli, Ciaran Ryan-Anderson, Dominic Lucchetti, Adam Paetznick, Martin Roetteler, Krysta Svore, and Alex Chernoguzov. 2023. Advances in compilation for quantum hardware – A demonstration of magic state distillation and repeat-until-success protocols. arXiv:2310.12106 [quant-ph] <https://arxiv.org/abs/2310.12106>
- [8] Lukas Burgholzer, Sarah Schneider, and Robert Wille. 2022. Limiting the Search Space in Optimal Quantum Circuit Mapping. In *2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, Taipei, Taiwan, 466–471. doi:10.1109/ASP-DAC52403.2022.9712555
- [9] Hugh GA Burton, Daniel Marti-Dafcik, David P Tew, and David J Wales. 2023. Exact electronic states with shallow quantum circuits from global optimisation. *npj Quantum Information* 9, 1 (2023), 75. <https://www.nature.com/articles/s41534-023-00744-2>
- [10] Fangzheng Chen, Hao Fu, Mingzheng Zhu, Chi Zhang, Wei Xie, and Xiang-Yang Li. 2026. A Framework for Dynamic Quantum Circuit Execution: Balancing Effectiveness and Efficiency. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 45, 6 (2026), 2583–2596. doi:10.1109/TCAD.2025.3626447
- [11] Xueyun Cheng, Zhijin Guan, and Pengcheng Zhu. 2020. Nearest Neighbor Transformation of Quantum Circuits in 2D Architecture. *IEEE Access* 8 (2020), 222466–222475. doi:10.1109/ACCESS.2020.3043497
- [12] A. D. Córcoles, Maika Takita, Ken Inoue, Scott Lekuch, Zlatko K. Mineev, Jerry M. Chow, and Jay M. Gambetta. 2021. Exploiting Dynamic Quantum Circuits in a Quantum Algorithm with Superconducting Qubits. *Phys. Rev. Lett.* 127 (Aug 2021), 100501. Issue 10. doi:10.1103/PhysRevLett.127.100501
- [13] Andrew Cross, Ali Javadi-Abhari, Thomas Alexander, Niel De Beaudrap, Lev S Bishop, Steven Heidel, Colm A Ryan, Prasanth Sivarajah, John Smolin, Jay M Gambetta, et al. 2022. OpenQASM 3: A broader and deeper quantum assembly language. *ACM Transactions on Quantum Computing* 3, 3 (2022), 1–50.
- [14] Michael Cubeddu, Will Finigan, Thomas Lively, Johannes Flick, and Prineha Narang. 2019. Introducing Control Flow in Qubit Allocation for Quantum Turing Machines. arXiv:1907.07113 [quant-ph] <https://arxiv.org/abs/1907.07113>
- [15] Samudra Dasgupta and Travis S. Humble. 2022. Adaptive stabilization of quantum circuits executed on unstable devices. In *2022 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, Broomfield, Colorado, USA, 736–740. doi:10.1109/QCE53715.2022.00102
- [16] Yongshan Ding, Xin-Chuan Wu, Adam Holmes, Ash Wiseth, Diana Franklin, Margaret Martonosi, and Frederic T. Chong. 2020. SQUARE: strategic quantum ancilla reuse for modular quantum programs via cost-effective uncomputation. In *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture (Virtual Event) (ISCA '20)*. IEEE Press, Virtual Event, 570–583. doi:10.1109/ISCA45697.2020.00054
- [17] Michael Foss-Feig, Arkin Tikku, Tsung-Cheng Lu, Karl Mayer, Mohsin Iqbal, Thomas M. Gatterman, Justin A. Gerber, Kevin Gilmore, Dan Gresh, Aaron Hankin, Nathan Hewitt, Chandler V. Horst, Mitchell Matheny, Tanner Mengle, Brian Neyenhuis, Henrik Dreyer, David Hayes, Timothy H. Hsieh, and Isaac H. Kim. 2023. Experimental demonstration of the advantage of adaptive quantum circuits. arXiv:2302.03029 [quant-ph] <https://arxiv.org/abs/2302.03029>
- [18] Blake Gerard, Tobias Grosser, and Martin Kong. 2022. QRANE: Lifting QASM Programs to an Affine IR. In *Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction (Seoul, South Korea) (CC 2022)*. Association for Computing Machinery, New York, NY, USA, 15–28. doi:10.1145/3497776.3517775
- [19] Craig Gidney. 2021. Stim: a fast stabilizer circuit simulator. *Quantum* 5 (2021), 497.

- [20] Rajiv Gupta. 1992. Generalized dominators and post-dominators. In *Proceedings of the 19th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Albuquerque, New Mexico, USA) (POPL '92). Association for Computing Machinery, New York, NY, USA, 246–257. doi:10.1145/143165.143216
- [21] Riddhi Swaroop Gupta, Luke C. G. Govia, and Michael J. Biercuk. 2020. Integration of spectator qubits into quantum computer architectures for hardware tune-up and calibration. *Phys. Rev. A* 102 (Oct 2020), 042611. Issue 4. doi:10.1103/PhysRevA.102.042611
- [22] Shay Hacothen-Gourgy and Leigh S Martin. 2020. Continuous measurements for control of superconducting quantum circuits. *Advances in Physics: X* 5, 1 (2020), 1813626. doi:10.1080/23746149.2020.1813626
- [23] Akel Hashim, Ming Yuan, Pranav Gokhale, Larry Chen, Christian Juenger, Neelay Fruitwala, Yilun Xu, Gang Huang, Kasra Nowrouzi, Liang Jiang, and Irfan Siddiqi. 2025. Efficient Generation of Multi-partite Entanglement between Non-local Superconducting Qubits using Classical Feedback. arXiv:2403.18768 [quant-ph] <https://arxiv.org/abs/2403.18768>
- [24] Fei Hua, Yuwei Jin, Yanhao Chen, Suhas Vittal, Kevin Krsulich, Lev S. Bishop, John Lapeyre, Ali Javadi-Abhari, and Eddy Z. Zhang. 2023. CaQR: A Compiler-Assisted Approach for Qubit Reuse through Dynamic Circuit. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (Vancouver, BC, Canada) (ASPLOS '23). Association for Computing Machinery, New York, NY, USA, 59–71. doi:10.1145/3582016.3582030
- [25] Keli Huang and Jens Palsberg. 2024. Compiling Conditional Quantum Gates without Using Helper Qubits. *Proc. ACM Program. Lang.* 8, PLDI, Article 206 (June 2024), 22 pages. doi:10.1145/3656436
- [26] Hanru Jiang. 2024. Qubit Recycling Revisited. *Proc. ACM Program. Lang.* 8, PLDI, Article 198 (June 2024), 24 pages. doi:10.1145/3656428
- [27] Hyungseok Kim, Enhyeok Jang, Seungwoo Choi, Youngmin Kim, and Won Woo Ro. 2025. QR-Map: A Map-Based Approach to Quantum Circuit Abstraction for Qubit Reuse Optimization. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery, New York, NY, USA, 1568–1582. doi:10.1145/3695053.3731020
- [28] Gushu Li, Yufei Ding, and Yuan Xie. 2019. Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) (ASPLOS '19). ACM, New York, NY, USA, 1001–1014. doi:10.1145/3297858.3304023
- [29] Matt Lourens, Ilya Sinayskiy, Daniel K Park, Carsten Blank, and Francesco Petruccione. 2023. Hierarchical quantum circuit representations for neural architecture search. *npj Quantum Information* 9, 1 (2023), 79.
- [30] Erika Magnusson, Aaron Fitzpatrick, Stefan Knecht, Martin Rahm, and Werner Dobrautz. 2024. Towards efficient quantum computing for quantum chemistry: Reducing circuit complexity with transcorrelated and adaptive ansatz techniques. *Faraday Discussions* 254 (2024), 402–428. <https://pubs.rsc.org/en/content/articlehtml/2024/fd/d4fd00039k>
- [31] Swarnadeep Majumder, Leonardo Andreta de Castro, and Kenneth R Brown. 2020. Real-time calibration with spectator qubits. *npj Quantum Information* 6, 1 (2020), 19. doi:10.1038/s41534-020-0251-y
- [32] Tillmann Miltzow, Lothar Narins, Yoshio Okamoto, Günter Rote, Antonis Thomas, and Takeaki Uno. 2016. Approximation and Hardness for Token Swapping. arXiv:1602.05150. arXiv preprint.
- [33] Abtin Molavi, Amanda Xu, Swamit Tannu, and Aws Albarghouthi. 2025. Dependency-Aware Compilation for Surface Code Quantum Architectures. *Proc. ACM Program. Lang.* 9, OOPSLA1, Article 82 (April 2025), 28 pages. doi:10.1145/3720416
- [34] Pranav Mundada, Gengyan Zhang, Thomas Hazard, and Andrew Houck. 2019. Suppression of qubit crosstalk in a tunable coupling superconducting circuit. *Physical Review Applied* 12, 5 (2019), 054023.
- [35] Prakash Murali, Jonathan M. Baker, Ali Javadi-Abhari, Frederic T. Chong, and Margaret Martonosi. 2019. Noise-Adaptive Compiler Mappings for Noisy Intermediate-Scale Quantum Computers. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) (ASPLOS '19). ACM, New York, NY, USA, 1015–1029. doi:10.1145/3297858.3304075
- [36] Siyuan Niu, Akel Hashim, Costin Iancu, Wibe Albert de Jong, and Ed Younis. 2024. Effective Quantum Resource Optimization via Circuit Resizing in BQSKit. In *Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC '24)*. Association for Computing Machinery, New York, NY, USA, 1–6. doi:10.1145/3649329.3656534
- [37] Siyuan Niu, Efehan Kokcu, Anupam Mitra, Aaron Szasz, Akel Hashim, Justin Kalloor, Wibe Albert de Jong, Costin Iancu, and Ed Younis. 2024. AC/DC: Automated Compilation for Dynamic Circuits. arXiv:2412.07969 [quant-ph] <https://arxiv.org/abs/2412.07969>
- [38] Siyuan Niu, Adrien Suau, Gabriel Staffelbach, and Aida Todri-Sanial. 2020. A Hardware-Aware Heuristic for the Qubit Mapping Problem in the NISQ Era. *IEEE Transactions on Quantum Engineering* 1 (2020), 1–14. doi:10.1109/TQE.2020.3026544
- [39] Anouk Paradis, Benjamin Bichsel, Samuel Steffen, and Martin Vechev. 2021. Unqomp: synthesizing uncomputation in Quantum circuits. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (Virtual, Canada) (PLDI 2021)*. Association for Computing Machinery, New York, NY, USA, 222–236. doi:10.1145/3453483.3454040
- [40] Sunghye Park, Daeyeon Kim, Minhyuk Kweon, Jae-Yoon Sim, and Seokhyeong Kang. 2022. A fast and scalable qubit-mapping method for noisy intermediate-scale quantum computers. In *Proceedings of the 59th ACM/IEEE Design Automation Conference* (San Francisco, California) (DAC '22). Association for Computing Machinery, New York, NY, USA, 13–18. doi:10.1145/3489517.3530402
- [41] Aditya Pawar, Yingheng Li, Zewei Mo, Yanan Guo, Xulong Tang, Youtao Zhang, and Jun Yang. 2024. QRCC: Evaluating Large Quantum Circuits on Small Quantum Computers through Integrated Qubit Reuse and Circuit Cutting. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4* (Hilton La Jolla Torrey Pines, La Jolla, CA, USA) (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 236–251. doi:10.1145/3622781.3674179
- [42] Tom Peham, Lukas Burgholzer, and Robert Wille. 2023. On Optimal Subarchitectures for Quantum Circuit Mapping. *ACM Transactions on Quantum Computing* 4, 4, Article 23 (July 2023), 20 pages. doi:10.1145/3593594
- [43] Juan M Pino, Jennifer M Dreiling, Caroline Figgatt, John P Gaebler, Steven A Moses, MS Allman, CH Baldwin, Michael Foss-Feig, David Hayes, Karl Mayer, et al. 2021. Demonstration of the trapped-ion quantum CCD computer architecture. *Nature* 592, 7853 (2021), 209–213. doi:10.1038/s41586-021-03318-4
- [44] Timothy Proctor, Melissa Revelle, Erik Nielsen, Kenneth Rudinger, Daniel Lobser, Peter Maunz, Robin Blume-Kohout, and Kevin Young. 2020. Detecting and tracking drift in quantum information processors. *Nature communications* 11, 1 (2020), 5396. doi:10.1038/s41467-020-19074-4
- [45] Vikram Ravindranath, Yiqiu Han, Zhi-Cheng Yang, and Xiao Chen. 2023. Entanglement steering in adaptive circuits with feedback. *Phys. Rev. B* 108 (Jul 2023), L041103. Issue 4. doi:10.1103/PhysRevB.108.L041103
- [46] Yuanjie Ren, Nathanan Tantivasadakarn, and Dominic J Williamson. 2025. Efficient preparation of solvable anyons with adaptive quantum circuits. *Physical Review X* 15, 3 (2025), 031060. <https://journals.aps.org/prx/pdf/10.1103/b9hf-gx4f>

- [47] Enrico Russo, Elio Vinciguerra, Maurizio Palesi, Davide Patti, Giuseppe Ascia, and Vincenzo Catania. 2025. TeleSABRE: Heuristic Layout Synthesis in Multi-Core Quantum Systems with Teleport Interconnect. In *2025 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Vol. 01. 749–758. doi:10.1109/QCE65121.2025.00086
- [48] Colm A. Ryan, Blake R. Johnson, Diego Ristè, Brian Donovan, and Thomas A. Ohki. 2017. Hardware for dynamic quantum computing. *Review of Scientific Instruments* 88, 10 (10 2017), 104703. doi:10.1063/1.5006525
- [49] Sokea Sang, Leanghok Hour, and Youngsun Han. 2026. Learning-optimized qubit mapping and reuse to minimize intercore communication in modular quantum architectures. *Physical Review Applied* 25, 5 (May 2026). doi:10.1103/rfn7-c5n6
- [50] K. Singh, C. E. Bradley, S. Anand, V. Ramesh, R. White, and H. Bernien. 2023. Mid-circuit correction of correlated phase errors using an array of spectator qubits. *Science* 380, 6651 (2023), 1265–1269. doi:10.1126/science.ade5337 arXiv:https://www.science.org/doi/pdf/10.1126/science.ade5337
- [51] Marcos Yukio Siraichi, Vinicius Fernandes dos Santos, Caroline Colange, and Fernando Magno Quintão Pereira. 2019. Qubit allocation as a combination of subgraph isomorphism and token swapping. *Proceedings of the ACM on Programming Languages* 3, OOPSLA (2019), 1–29.
- [52] Marcos Yukio Siraichi, Vinicius Fernandes dos Santos, Sylvain Colange, and Fernando Magno Quintao Pereira. 2018. Qubit Allocation. In *Proceedings of the 2018 International Symposium on Code Generation and Optimization (Vienna, Austria) (CGO 2018)*. ACM, New York, NY, USA, 113–125. doi:10.1145/3168822
- [53] Seyon Sivarajah, Silas Dilkes, Alexander Cowtan, Will Simmons, Alec Edgington, and Ross Duncan. 2020. tket: a retargetable compiler for NISQ devices. *Quantum Science and Technology* 6, 1 (11 2020), 014003. doi:10.1088/2058-9565/ab8e92
- [54] Kevin C. Smith, Abid Khan, Bryan K. Clark, S.M. Girvin, and Tzu-Chieh Wei. 2024. Constant-Depth Preparation of Matrix Product States with Adaptive Quantum Circuits. *PRX Quantum* 5 (Sep 2024), 030344. Issue 3. doi:10.1103/PRXQuantum.5.030344
- [55] Bochen Tan and Jason Cong. 2020. Optimal layout synthesis for quantum computing. In *International Conference On Computer Aided Design (ICCAD)*. ACM, San Diego, California, USA, 1–9.
- [56] Bochen Tan and Jason Cong. 2020. Optimality Study of Existing Quantum Computing Layout Synthesis Tools. *IEEE Trans. Comput.* 70, 9 (2020), 1363–1373. doi:10.1109/TC.2020.3009140
- [57] Ho Lun Tang, V.O. Shkolnikov, George S. Barron, Harper R. Grimley, Nicholas J. Mayhall, Edwin Barnes, and Sophia E. Economou. 2021. Qubit-ADAPT-VQE: An Adaptive Algorithm for Constructing Hardware-Efficient Ansätze on a Quantum Processor. *PRX Quantum* 2 (Apr 2021), 020310. Issue 2. doi:10.1103/PRXQuantum.2.020310
- [58] Hristo Venev, Timon Gehr, Dimitar Dimitrov, and Martin Vechev. 2024. Modular Synthesis of Efficient Quantum Uncomputation. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 345 (Oct. 2024), 28 pages. doi:10.1145/3689785
- [59] Sven Verdoolaege. 2010. isl: An Integer Set Library for the Polyhedral Model. In *ICMS*, Vol. 6327. Springer, Springer, Kobe, Japan, 299–302.
- [60] Robert Wille and Lukas Burgholzer. 2023. MQT QMAP: Efficient Quantum Circuit Mapping. In *Proceedings of the 2023 International Symposium on Physical Design (Virtual Event, USA) (ISPD '23)*. Association for Computing Machinery, New York, NY, USA, 198–204. doi:10.1145/3569052.3578928
- [61] Robert Wille, Lukas Burgholzer, and Alwin Zulehner. 2019. Mapping quantum circuits to IBM QX architectures using the minimal number of SWAP and H operations. In *2019 56th ACM/IEEE Design Automation Conference (DAC)*. IEEE, IEEE, Europe, 1–6.
- [62] Robert Wille, Stefan Hillmich, and Lukas Burgholzer. 2020. JKQ: JKU Tools for Quantum Computing. In *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, Europe, 1–5. doi:10.1145/3400302.3415746
- [63] Robert Wille, Oliver Keszocze, Marcel Walter, Patrick Rohrs, Anupam Chattopadhyay, and Rolf Drechsler. 2016. Look-ahead schemes for nearest neighbor optimization of 1D and 2D quantum circuits. In *Design Automation Conference (ASP-DAC), 2016 21st Asia and South Pacific*. IEEE, IEEE, Asia and South Pacific, 292–297.
- [64] Robert Wille, Aaron Lye, and Rolf Drechsler. 2014. Optimal SWAP gate insertion for nearest neighbor quantum circuits. In *Design Automation Conference (ASP-DAC), 2014 19th Asia and South Pacific*. IEEE, IEEE, Asia and South Pacific, 489–494.
- [65] Xin-Chuan Wu, Marc Grau Davis, Frederic T. Chong, and Costin Iancu. 2021. Reoptimization of Quantum Circuits via Hierarchical Synthesis. In *2021 International Conference on Rebooting Computing (ICRC)*. 35–46. doi:10.1109/ICRC53822.2021.00016
- [66] Nikola Yanakiev, Normann Mertig, Christopher K. Long, and David R. M. Arvidsson-Shukur. 2024. Dynamic adaptive quantum approximate optimization algorithm for shallow, noise-resilient circuits. *Phys. Rev. A* 109 (Mar 2024), 032420. Issue 3. doi:10.1103/PhysRevA.109.032420
- [67] Hezi Zhang, Keyi Yin, Anbang Wu, Hassan Shapourian, Alireza Shabani, and Yufei Ding. 2024. Mech: Multi-entry communication highway for superconducting quantum chiplets. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. ACM, USA, 699–714.
- [68] Henry Zou, Matthew Treinish, Kevin Hartman, Alexander Ivrii, and Jake Lishman. 2024. LightSABRE: A Lightweight and Enhanced SABRE Algorithm. *arXiv preprint arXiv:2409.08368* (2024). https://arxiv.org/abs/2409.08368
- [69] Alwin Zulehner, Alexandru Paler, and Robert Wille. 2018. Efficient mapping of quantum circuits to the IBM QX architectures. In *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2018*. IEEE, IEEE, Europe, 1135–1138.